

RAPPORT DE STAGE D'ÉTÉ

Conception, réalisation et déploiement d'une plateforme de suivi et d'évaluation de la bonne gouvernance

Projet SSE — Système de Suivi et d'Évaluation

RÉALISÉ PAR **Khelifi Mohamed Anas**

FORMATION **1ère année du cycle ingénieur**

ORGANISME D'ACCUEIL **CNI — Centre National de l'Informatique**

ENCADRANT ENTREPRISE **M. Imed Zaier**

PÉRIODE **Du 12 juin 2026 au 30 juillet 2026**

Année universitaire 2025–2026

Remerciements

Je tiens à exprimer ma profonde gratitude à l'ensemble des personnes qui ont contribué au bon déroulement de ce stage d'été et à la réalisation du projet présenté dans ce rapport.

Je remercie tout particulièrement M. Imed Zaier, mon encadrant au Centre National de l'Informatique, pour son accueil, sa disponibilité et ses orientations. Ses retours m'ont permis de mieux comprendre les exigences d'une application institutionnelle et d'adopter une démarche de travail structurée, attentive à la sécurité, à la traçabilité et à l'expérience des utilisateurs.

Mes remerciements s'adressent également aux équipes du CNI pour le cadre professionnel offert, ainsi qu'aux enseignants d'ESPRIT pour les connaissances méthodologiques et techniques mobilisées pendant cette expérience. Enfin, je remercie ma famille et mes proches pour leur soutien constant.

Résumé

Ce rapport présente la conception, la réalisation et le déploiement d'une plateforme web de suivi et d'évaluation de la bonne gouvernance, développée dans le cadre d'un stage d'été effectué au Centre National de l'Informatique. La solution, appelée SSE, structure un processus complet allant de la création des comptes et des organismes jusqu'à la validation des évaluations, au calcul des scores et à la consultation d'indicateurs gouvernementaux.

La plateforme repose sur une architecture en couches. Le front-end est développé avec React et TypeScript, tandis que le back-end utilise Spring Boot et expose une API REST sécurisée par des jetons JWT. PostgreSQL assure la persistance des données. L'ensemble est conteneurisé avec Docker Compose et déployé sur une instance Ubuntu dans Amazon EC2.

Le travail a porté sur la modélisation fonctionnelle et UML, la consolidation du workflow d'évaluation, l'amélioration de l'interface et de l'expérience utilisateur, la gestion multilingue propre à chaque compte, l'ajout de justificatifs, les notifications, les exports, les contrôles d'accès et la traçabilité. Une attention particulière a été accordée aux parcours des quatre profils : administrateur, utilisateur, évaluateur et gouvernement.

MOTS-CLÉS bonne gouvernance, évaluation, Spring Boot, React, PostgreSQL, JWT, Docker, AWS EC2, expérience utilisateur, UML

Fiche signalétique du stage

Rubrique	Information
Stagiaire	Khelifl Mohamed Anas
Établissement	ESPRIT — École Supérieure Privée d'Ingénierie et de Technologies
Niveau	1ère année du cycle ingénieur
Année universitaire	2025–2026
Nature du stage	Stage d'été
Organisme d'accueil	CNI — Centre National de l'Informatique
Encadrant entreprise	M. Imed Zaier
Encadrant académique	À compléter par l'établissement
Période	12 juin 2026 – 30 juillet 2026
Sujet	Conception, réalisation et déploiement d'une plateforme de suivi et d'évaluation de la bonne gouvernance

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
AWS	Amazon Web Services
CNI	Centre National de l'Informatique
CORS	Cross-Origin Resource Sharing
DTO	Data Transfer Object
EC2	Elastic Compute Cloud
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JWT	JSON Web Token
REST	Representational State Transfer
SSE	Système de Suivi et d'Évaluation
UML	Unified Modeling Language
UX / UI	User Experience / User Interface

Table des matières

Introduction générale.....	1
Problématique.....	1
Objectifs du stage.....	1
Chapitre 1 — Cadre du stage et étude de l'existant	2
1.1 Présentation d'ESPRIT	2
1.2 Présentation du Centre National de l'Informatique.....	2
1.3 Cadre et sujet du stage.....	2
1.4 Analyse de l'existant.....	3
1.5 Démarche de travail	3
1.6 Résultats attendus.....	3
Chapitre 2 — Spécification des besoins	4
2.1 Identification des acteurs	4
2.2 Besoins fonctionnels.....	4
2.3 Diagramme de cas d'utilisation	5
2.4 Règles de gestion.....	7
2.5 Besoins non fonctionnels	7
2.6 Priorisation	8
Chapitre 3 — Conception de la solution	9
3.1 Architecture applicative	9
3.2 Conception de la sécurité.....	11
3.3 Modèle du domaine	12
3.4 Cycle de vie des évaluations.....	14
3.5 Calcul des scores et de la maturité.....	14
3.6 Diagrammes de séquence	14
3.7 Choix de conception complémentaires.....	19
Chapitre 4 — Réalisation et améliorations apportées	20
4.1 Environnement technique.....	20
4.2 Réalisation du back-end	20
4.3 Refonte de l'interface et de l'expérience utilisateur.....	21
4.4 Internationalisation propre à chaque compte	24
4.5 Paramètres utiles	24
4.6 Qualité d'usage.....	25

Chapitre 5 — Tests, déploiement et exploitation	26
5.1 Stratégie de test	26
5.2 Cas de test critiques	26
5.3 Conteneurisation	27
5.4 Déploiement sur Ubuntu dans AWS EC2.....	27
5.5 Sécurité d'exploitation	29
5.6 Résultats de vérification	29
Chapitre 6 — Bilan et perspectives	30
6.1 Difficultés rencontrées et solutions	30
6.2 Compétences acquises	30
6.3 Apports personnels au projet.....	30
6.4 Limites actuelles	31
6.5 Perspectives d'évolution	31
Conclusion générale	32
Bibliographie et webographie	33
Annexes	34
Annexe A — Principaux points d'entrée REST.....	34
Annexe B — Matrice des transitions	34
Annexe C — Installation locale synthétique.....	35
Annexe D — Glossaire métier	35
Annexe E — Livrables UML.....	35
Annexe F — Checklist avant mise en production.....	36

Liste des figures

- Figure 1 — Diagramme de cas d'utilisation de la plateforme SSE
- Figure 2 — Architecture applicative en couches
- Figure 3 — Diagramme de classes du domaine SSE
- Figures 4 à 7 — Séquences de remplissage, soumission, validation et correction
- Figure 8 — Tableau de bord administrateur
- Figure 9 — Tableau de bord utilisateur
- Figure 10 — File de validation de l'évaluateur
- Figure 11 — Interface de revue des critères
- Figure 12 — Tableau de bord gouvernemental
- Figure 13 — Architecture de déploiement

Introduction générale

La transformation numérique des administrations et des organismes publics ne se limite plus à la dématérialisation de formulaires. Elle implique la mise en place de systèmes capables de structurer l'information, de sécuriser les échanges, d'assurer la traçabilité des décisions et de produire des indicateurs fiables. Dans ce contexte, l'évaluation de la bonne gouvernance constitue un exercice particulièrement exigeant : elle combine un référentiel détaillé, des preuves documentaires, une appréciation par critère, des échanges de correction et une décision finale.

Le projet SSE répond à ce besoin par une plateforme web multi-profils. Un organisme peut renseigner son auto-évaluation et joindre les justificatifs nécessaires. Un évaluateur examine chaque réponse, valide, rejette ou demande une correction. L'administrateur supervise les comptes, le référentiel, les évaluations et les opérations système. Le profil gouvernement dispose, quant à lui, d'une vue consolidée orientée vers les indicateurs et le classement, sans intervenir dans le remplissage des évaluations.

Le stage réalisé au Centre National de l'Informatique a consisté à analyser ce domaine, à consolider une base applicative existante, à corriger plusieurs défauts fonctionnels et visuels, à améliorer les parcours utilisateurs et à préparer une version déployable. La démarche adoptée a combiné étude du code, modélisation UML, développement itératif, tests ciblés, vérification sur l'environnement déployé et documentation.

Problématique

QUESTION DIRECTRICE Comment concevoir une plateforme institutionnelle qui rende l'évaluation de la bonne gouvernance à la fois rigoureuse, traçable, sécurisée et simple à utiliser pour des profils aux responsabilités très différentes ?

Cette problématique se décline en plusieurs enjeux. Le référentiel doit être maintenable et multilingue. Le workflow doit empêcher une validation incomplète ou concurrente. Les preuves doivent être rattachées au bon critère. Les données d'un organisme doivent rester isolées de celles des autres. Enfin, l'interface doit rester lisible malgré un volume important de principes, de bonnes pratiques et de critères.

Objectifs du stage

- formaliser le métier, ses acteurs, ses règles et ses états ;
- stabiliser l'architecture full-stack et sécuriser l'accès aux ressources ;
- fluidifier le cycle remplir – soumettre – examiner – corriger – valider ;
- réaliser une refonte UI/UX cohérente, responsive et multilingue ;
- tester les parcours critiques, vérifier le déploiement et documenter la solution.

Le **rapport** suit le cheminement du projet — contexte et besoins, conception, réalisation, tests et déploiement — avant de présenter mon bilan et les évolutions encore possibles.

Chapitre 1 — Cadre du stage et étude de l'existant

Ce chapitre situe le stage dans son environnement académique et professionnel. Il présente l'établissement de formation, l'organisme d'accueil, le contexte du projet SSE et la démarche suivie pour transformer une base applicative en une solution cohérente, testable et déployable.

1.1 Présentation d'ESPRIT

ESPRIT est un établissement d'enseignement supérieur spécialisé dans les domaines de l'ingénierie et des technologies. L'école met en avant une pédagogie active fondée sur les projets et les problèmes, qui place l'étudiant dans des situations proches du travail en entreprise. Cette approche encourage l'autonomie, la collaboration, l'analyse des besoins et la production de livrables vérifiables [3].

La première année du cycle ingénieur consolide les bases scientifiques et techniques tout en développant la capacité à structurer un projet. Le stage d'été complète cette formation par une immersion professionnelle. Il permet de confronter les acquis académiques à des contraintes réelles : continuité d'un code existant, sécurité, qualité de données, expérience utilisateur, déploiement et communication avec un encadrant.

1.2 Présentation du Centre National de l'Informatique

Le Centre National de l'Informatique est un établissement public tunisien créé le 30 décembre 1975. Doté de la personnalité civile et de l'autonomie financière, il est placé sous la tutelle du ministère chargé des technologies de la communication. Sa mission générale consiste à accompagner l'administration et les organismes publics dans l'utilisation des technologies de l'information [1].

Le CNI intervient dans des domaines qui exigent fiabilité, continuité de service et gouvernance des données. Son site institutionnel présente notamment des activités d'accompagnement, de formation et d'accueil de stagiaires. Le centre indique recevoir environ deux cents stagiaires par an, ce qui témoigne d'un rôle actif dans l'insertion et le développement des compétences des étudiants [2].

REPÈRE INSTITUTIONNEL Le CNI annonce une démarche qualité certifiée ISO 9001:2015. Cette culture de processus et de traçabilité est cohérente avec la nature du projet SSE, qui formalise les étapes d'évaluation et conserve l'historique des actions [1].

1.3 Cadre et sujet du stage

Le stage s'est déroulé du 12 juin au 30 juillet 2026 sous l'encadrement de M. Imed Zaier. Le sujet retenu porte sur une plateforme de suivi et d'évaluation de la bonne gouvernance. L'application doit soutenir plusieurs usages : administrer les organismes et les comptes, guider l'auto-évaluation, centraliser les preuves, organiser la revue par un évaluateur, calculer des scores et présenter une vue de pilotage.

Le projet existait sous la forme d'un dépôt full-stack comprenant un back-end Spring Boot, un front-end React et une configuration Docker. L'étude initiale a montré que les composants essentiels étaient présents, mais que plusieurs points devaient être consolidés : cohérence des rôles, sécurité des ressources, transitions de statut, concurrence entre évaluateurs, traduction, formulaires incomplets, défauts d'affichage et déploiement reproductible.

1.4 Analyse de l'existant

Domaine	État observé	Besoin de consolidation
Architecture	Application React, API Spring Boot et PostgreSQL déjà structurées.	Clarifier les responsabilités des couches et fiabiliser la configuration par environnement.
Authentification	Connexion JWT et rôles disponibles.	Renforcer les contrôles d'accès sur les ressources appartenant aux organismes.
Évaluation	Principes, bonnes pratiques, critères et réponses modélisés.	Sécuriser les transitions, la complétude, le verrou de validation et les corrections ciblées.
Interface	Écrans fonctionnels mais hétérogènes.	Améliorer navigation, responsive design, modales, formulaires, contrastes et feedbacks.
Internationalisation	Français, arabe et anglais présents.	Rendre la langue persistante par compte et gérer correctement la direction RTL.
Exploitation	Docker Compose disponible.	Valider la chaîne de construction et le déploiement sur Ubuntu/EC2.

1.5 Démarche de travail

La démarche adoptée est incrémentale. Chaque évolution commence par l'observation du comportement ou la lecture du code concerné, puis par l'identification de la règle métier. La modification est ensuite réalisée sur une branche dédiée, vérifiée localement et, pour les parcours visuels, contrôlée sur l'application déployée. Les versions importantes sont sauvegardées dans Git afin de conserver un point de retour.

Période	Phase	Livrables principaux
12–19 juin	Prise en main	Inventaire du dépôt, étude des rôles, du modèle et des parcours.
20–30 juin	Spécification et conception	Besoins, règles métier, diagrammes UML et architecture cible.
1–10 juillet	Consolidation back-end	Sécurité, workflow, scoring, notifications, audit et tests.
11–22 juillet	Refonte front-end	Navigation, interfaces par rôle, responsive design, i18n et formulaires.
23–27 juillet	Intégration et déploiement	Docker Compose, vérifications sur Ubuntu/EC2 et corrections finales.
28–30 juillet	Documentation et transfert	Rapport, synthèse technique et recommandations d'exploitation.

1.6 Résultats attendus

- une application cohérente pour les quatre rôles définis ;
- un cycle d'évaluation contrôlé, avec justificatifs et corrections ciblées ;
- des indicateurs et exports exploitables après validation ;
- une interface moderne, lisible sur différentes tailles d'écran et disponible en plusieurs langues ;
- une version conteneurisée, sauvegardée dans Git et déployable sur un serveur Ubuntu.

Le chapitre suivant traduit ces résultats attendus en spécifications fonctionnelles et non fonctionnelles.

Chapitre 2 — Spécification des besoins

La spécification vise à établir un langage commun entre le métier et la réalisation technique. Elle délimite les responsabilités de chaque acteur, les cas d'utilisation, les règles de gestion et les qualités attendues de la solution.

2.1 Identification des acteurs

Acteur	Responsabilités	Limites principales
Administrateur	Gère les utilisateurs, organismes, demandes de compte, référentiel, évaluations, notifications, réclamations, audit et courriels.	N'agit pas à la place d'un organisme pour produire ses preuves, sauf opération exceptionnelle contrôlée.
Utilisateur	Représente un organisme, crée ou ouvre une évaluation, renseigne les niveaux, commentaires et justificatifs, puis soumet ou corrige.	N'accède qu'aux données de son organisme et ne valide pas les résultats.
Évaluateur	Consulte la file de travail, prend une évaluation en charge, examine les réponses, valide, rejette ou demande une correction.	Ne modifie pas l'identité des organismes ni le référentiel administratif.
Gouvernement	Consulte les indicateurs consolidés et le classement des organismes évalués.	Ne remplit pas et ne valide pas les évaluations.

RÈGLE COMMUNE Les quatre acteurs peuvent se connecter à l'application. La navigation et les données visibles sont ensuite adaptées au rôle et aux autorisations du compte.

2.2 Besoins fonctionnels

2.2.1 Gestion des comptes et organismes

- permettre à un organisme de déposer une demande de compte complète, incluant coordonnées, secteur, fonction, télécopie et logo ;
- permettre à l'administrateur d'examiner, d'approuver ou de rejeter une demande ;
- créer l'organisme et son utilisateur principal après approbation, puis envoyer un parcours d'activation ;
- gérer l'état actif, le statut, le rôle et la version de jeton d'un utilisateur ;
- agrandir le logo d'un organisme depuis les listes ou les fiches détaillées afin d'en faciliter la vérification.

2.2.2 Gestion du référentiel

Le référentiel est organisé en principes, bonnes pratiques et critères. Chaque élément peut disposer de libellés dans plusieurs langues. Les critères contiennent les indications de preuves attendues et les références utiles. L'administrateur doit pouvoir maintenir ce référentiel sans modifier le code de l'application.

2.2.3 Cycle d'évaluation

- créer au maximum une évaluation pertinente par organisme et par année selon les règles métier ;
- sauvegarder progressivement le niveau N0 à N3, le commentaire, les liens et les fichiers justificatifs de chaque critère ;
- vérifier la complétude avant soumission et signaler précisément les éléments manquants ;

- autoriser un évaluateur à prendre en charge une validation et empêcher deux personnes de modifier simultanément le même dossier ;
- enregistrer une décision par réponse : validée, rejetée ou à corriger ;
- rouvrir uniquement les critères ciblés, conserver le motif et contrôler leur traitement avant resoumission ;
- calculer les scores et la maturité uniquement lorsque toutes les décisions nécessaires sont prises.

2.2.4 Pilotage et fonctions transverses

- afficher des tableaux de bord adaptés à chaque rôle ;
- notifier les acteurs lors des soumissions, corrections et validations ;
- proposer une diffusion en temps réel des notifications par flux SSE ;
- exporter les résultats en PDF et certaines données en Excel ;
- journaliser les opérations sensibles et suivre la file de courriels ;
- gérer les réclamations et les réponses administratives ;
- mémoriser la langue par compte, indépendamment du compte précédemment ouvert dans le même navigateur.

2.3 Diagramme de cas d'utilisation

Le diagramme suivant présente les quatre acteurs et les fonctionnalités essentielles. Les relations « include » représentent un comportement obligatoire réutilisé, par exemple le contrôle de complétude lors d'une soumission. Les relations « extend » représentent un scénario conditionnel, comme l'ajout d'un justificatif ou la demande de correction.

DIAGRAMME DE CAS D'UTILISATION

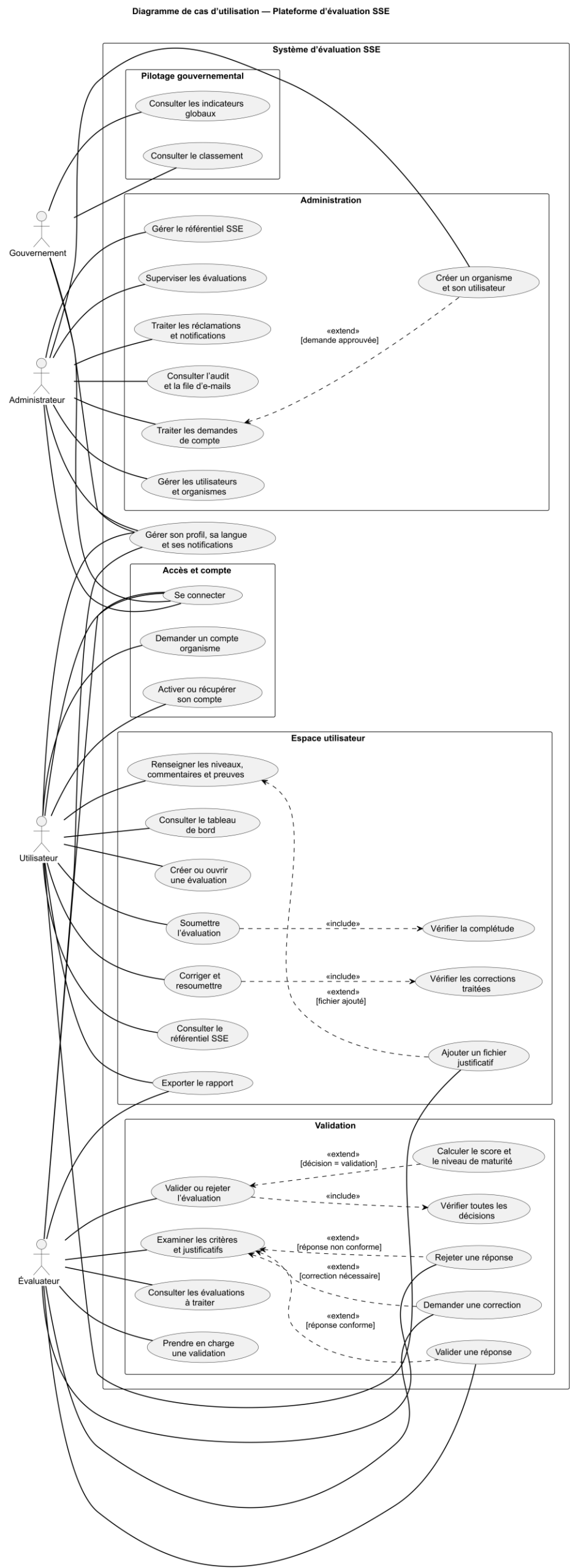


Figure 1 — Diagramme de cas d'utilisation officiel de la plateforme SSE.

2.4 Règles de gestion

Code	Règle
RG-01	Toute ressource protégée nécessite un compte authentifié et actif.
RG-02	Un utilisateur ne consulte et ne modifie que l'organisme auquel son compte est rattaché.
RG-03	Une évaluation ne peut être soumise que depuis l'état EN_COURS et lorsque les critères requis sont renseignés.
RG-04	Une évaluation soumise peut être prise en charge par un seul validateur à la fois.
RG-05	La validation finale est refusée tant qu'une réponse n'a pas reçu de décision ou qu'une correction reste demandée.
RG-06	Une demande de correction remet l'évaluation à EN_COURS et rouvre les critères ciblés.
RG-07	Le score global est calculé à partir des scores par principe et de leur pondération effective.
RG-08	Le profil gouvernement dispose de fonctions de lecture et de pilotage, sans accès au remplissage.
RG-09	Les opérations sensibles sont tracées dans le journal d'audit.
RG-10	Les fichiers envoyés doivent respecter la taille maximale et être rattachés à une réponse autorisée.

2.5 Besoins non fonctionnels

Qualité	Exigence retenue
Sécurité	JWT à durée limitée, mot de passe chiffré, autorisations par rôle et par propriétaire, CORS configuré, secrets externalisés.
Ergonomie	Navigation claire, retours explicites, boutons cohérents, modales sans défaut de voile, formulaires fermables et parcours réversibles.
Responsive design	Adaptation aux écrans courants, notamment pour la liste des principes, des bonnes pratiques et des critères.
Internationalisation	Français, arabe et anglais, prise en charge RTL et préférence propre à chaque compte.
Performance	Pagination des listes, requêtes ciblées, chargements différés et images dimensionnées.
Fiabilité	Transitions de statut contrôlées, transactions, verrou de validation, volumes persistants et healthchecks.
Maintenabilité	Séparation front-end/back-end, services métier, DTO, composants réutilisables, configuration par variables d'environnement.
Traçabilité	Journal d'audit, dates métier, identifiant du validateur et historique de notification.

2.6 Priorisation

La priorité a été donnée aux éléments qui conditionnent l'intégrité du processus. La sécurité des ressources, le workflow et la correction ciblée ont été traités avant les améliorations visuelles. La refonte UI/UX a ensuite été menée sur les parcours les plus utilisés, puis étendue aux écrans secondaires.

Priorité	Éléments
Must	Authentification, contrôle d'accès, remplissage, preuves, soumission, validation, correction, scoring, persistance.
Should	Notifications, exports, audit, formulaires complets, langue par compte, responsive design.
Could	Assistant contextuel, enrichissement des paramètres, indicateurs supplémentaires, automatisation CI/CD.
Hors périmètre immédiat	Application mobile native et connexion à des systèmes externes non spécifiés.

Cette priorisation a orienté mes choix tout au long du stage. Lorsqu'un défaut d'affichage révélait aussi un risque métier — par exemple une validation possible malgré des décisions incomplètes — j'ai traité d'abord la règle côté serveur, puis le retour visuel côté interface.

À l'inverse, les améliorations de confort sans incidence sur les données ont été regroupées dans la refonte UI/UX. Cette manière de travailler a évité de confondre urgence visuelle et criticité fonctionnelle.

Chapitre 3 — Conception de la solution

La conception vise à transformer les besoins en une structure logicielle explicite. Elle traite l'architecture en couches, le modèle du domaine, les mécanismes de sécurité, le calcul des scores et les interactions principales entre l'interface, l'API et la base de données.

3.1 Architecture applicative

La plateforme adopte une architecture client–serveur. Le navigateur exécute l'application React et communique avec une API REST Spring Boot. Le back-end concentre les règles métier, l'autorisation, les transactions et la génération de documents. PostgreSQL conserve les données structurées, tandis qu'un espace de fichiers persistant reçoit les justificatifs.

Au début du travail, j'ai choisi de ne pas mélanger les corrections d'interface avec les règles métier. Cette séparation m'a permis de vérifier une couche à la fois et de comprendre plus facilement l'origine d'une erreur : affichage côté React, contrat HTTP, règle de service ou persistance.

Principe directeur	Application dans SSE
Séparation des responsabilités	Composants React, contrôleurs REST, services métier et dépôts JPA distincts.
API sans état	Chaque requête authentifiée porte son jeton ; le serveur ne dépend pas d'une session HTTP.
Configuration externalisée	Base, CORS, JWT, SMTP et limites de fichiers pilotés par l'environnement.
Persistance explicite	Données relationnelles dans PostgreSQL et justificatifs dans un volume dédié.
Observabilité minimale	Healthchecks, audit, notifications et file de courriels consultables.

ARCHITECTURE APPLICATIVE

Architecture applicative de la plateforme SSE

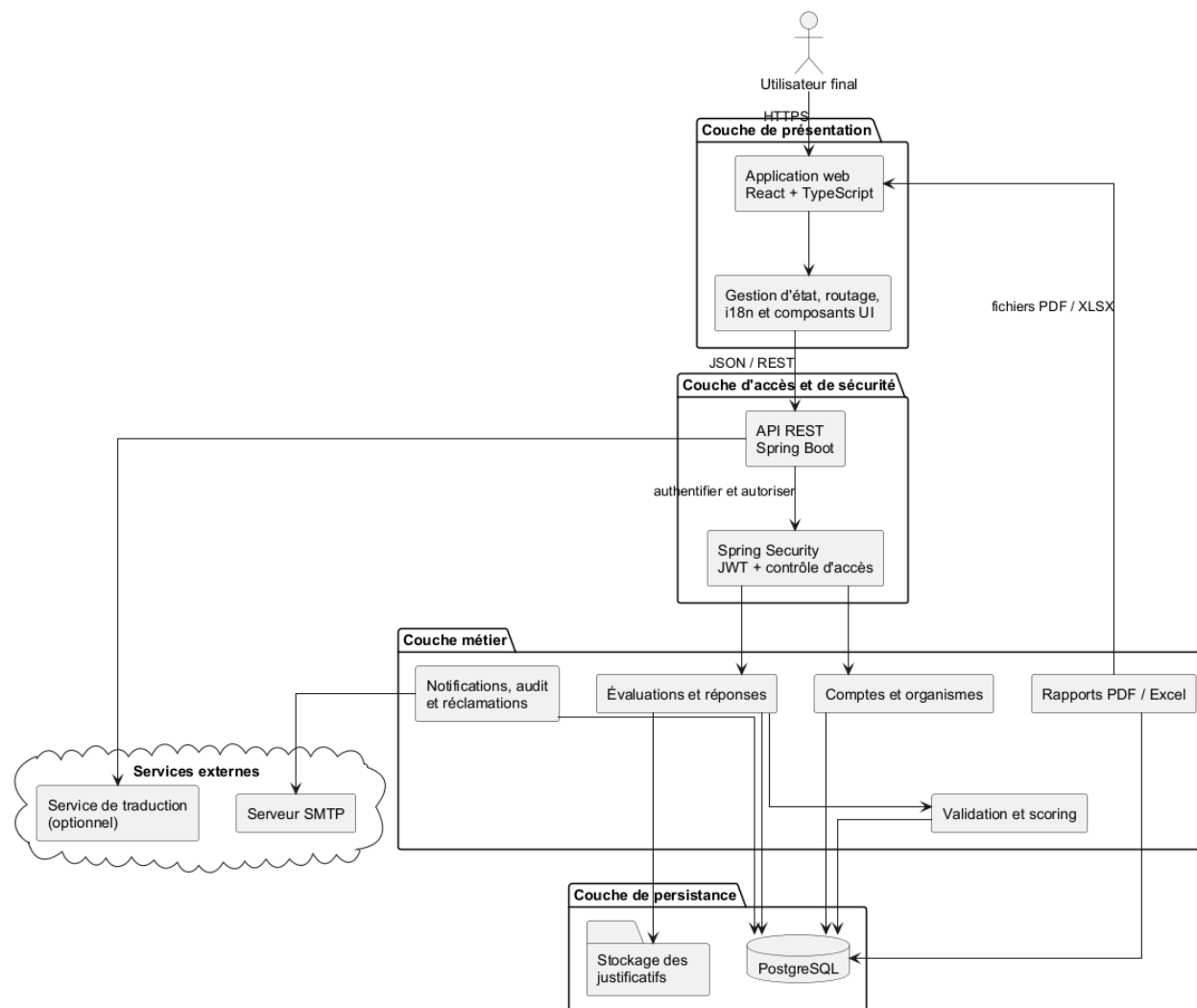


Figure 2 — Architecture applicative en couches de la plateforme SSE.

3.1.1 Couche de présentation

Le front-end est une application monopage construite avec React 18 et TypeScript. React Router organise les routes, Zustand conserve les états partagés utiles, React Hook Form et Zod structurent les formulaires, i18next gère les traductions et Recharts affiche les indicateurs. Axios centralise les appels à l'API et l'intercepteur d'authentification joint les jetons nécessaires.

3.1.2 Couche d'accès et métier

Le back-end, basé sur Spring Boot 3.2.5 et Java 17, expose des contrôleurs REST. Les contrôleurs valident la forme des requêtes et délèguent aux services. Les services appliquent les règles de gestion : création d'une évaluation, soumission, prise en charge, correction, validation, scoring, notifications, activation des comptes et audit. Des DTO limitent les informations échangées et évitent d'exposer directement le modèle de persistance.

3.1.3 Couche de persistance

Spring Data JPA assure l'accès à PostgreSQL. Les dépôts encapsulent les requêtes, notamment celles qui contrôlent l'appartenance d'une évaluation à un organisme ou récupèrent les réponses d'un principe. Les opérations sensibles sont transactionnelles afin d'éviter qu'une modification partielle laisse le dossier dans un état incohérent.

3.2 Conception de la sécurité

L'authentification est sans état. Après une connexion réussie, l'API renvoie un jeton d'accès JWT d'une durée courte et un mécanisme de renouvellement. Chaque requête protégée traverse un filtre qui extrait le jeton, vérifie sa signature et charge l'utilisateur. Spring Security applique ensuite les autorisations au niveau des routes et des méthodes.

Contrôle	Mécanisme
Authentification	Jetons JWT signés ; accès court de 15 minutes et renouvellement configuré jusqu'à 7 jours.
Autorisation par rôle	ADMIN, USER, EVALUATEUR et GOUVERNEMENT, avec hiérarchie contrôlée pour certaines actions.
Autorisation par ressource	Vérification de l'organisme propriétaire avant lecture, écriture ou téléversement d'un justificatif.
Mots de passe	Hachage via le composant PasswordEncoder ; aucun mot de passe en clair dans la base.
Révocation	Version de jeton et état du compte permettant d'invalidiser une session.
Configuration	Origines CORS, secrets JWT, base de données et SMTP externalisés dans l'environnement.
Traçabilité	Journalisation des soumissions, validations, rejets et opérations d'administration.

PRINCIPE DU MOINDRE PRIVILÈGE Le rôle donne une autorisation générale, mais l'accès à une évaluation ou à une réponse est aussi conditionné par la relation avec l'organisme. Cette double vérification limite les accès horizontaux entre organismes.

3.3 Modèle du domaine

Le cœur métier est formé par Organisme, Évaluation, Principe, Bonne pratique, Critère et Réponse. Une évaluation appartient à un organisme et contient des réponses associées aux critères du référentiel. Les scores par principe et le score global sont produits après la revue. Des entités de support gèrent les demandes de compte, notifications, réclamations, jetons d'activation, tâches de courriel et journaux d'audit.

Le diagramme de classes utilise des noms métier en français. Les types techniques des attributs — String, Integer, Boolean, UUID, LocalDateTime, List<String> — restent en anglais conformément aux conventions des langages de programmation et à la demande de cohérence avec le code source.

Agrégat	Responsabilité	Relations essentielles
Organisme	Porte l'identité et les coordonnées de la structure évaluée.	Utilisateurs, évaluations, scores et réclamations.
Référentiel	Décrit les principes, bonnes pratiques, critères, preuves attendues et traductions.	Principe → Bonne pratique → Critère.
Évaluation	Conserve l'année, le statut, les dates, le verrou, le score et la maturité.	Organisme, réponses et scores par principe.
Réponse	Associe à un critère un niveau, un commentaire, des justificatifs et une décision.	Évaluation, critère et validateur.
Support	Gère activation, notifications, courriels, réclamations et audit.	Utilisateur et entités métier concernées.

DIAGRAMME DE CLASSES

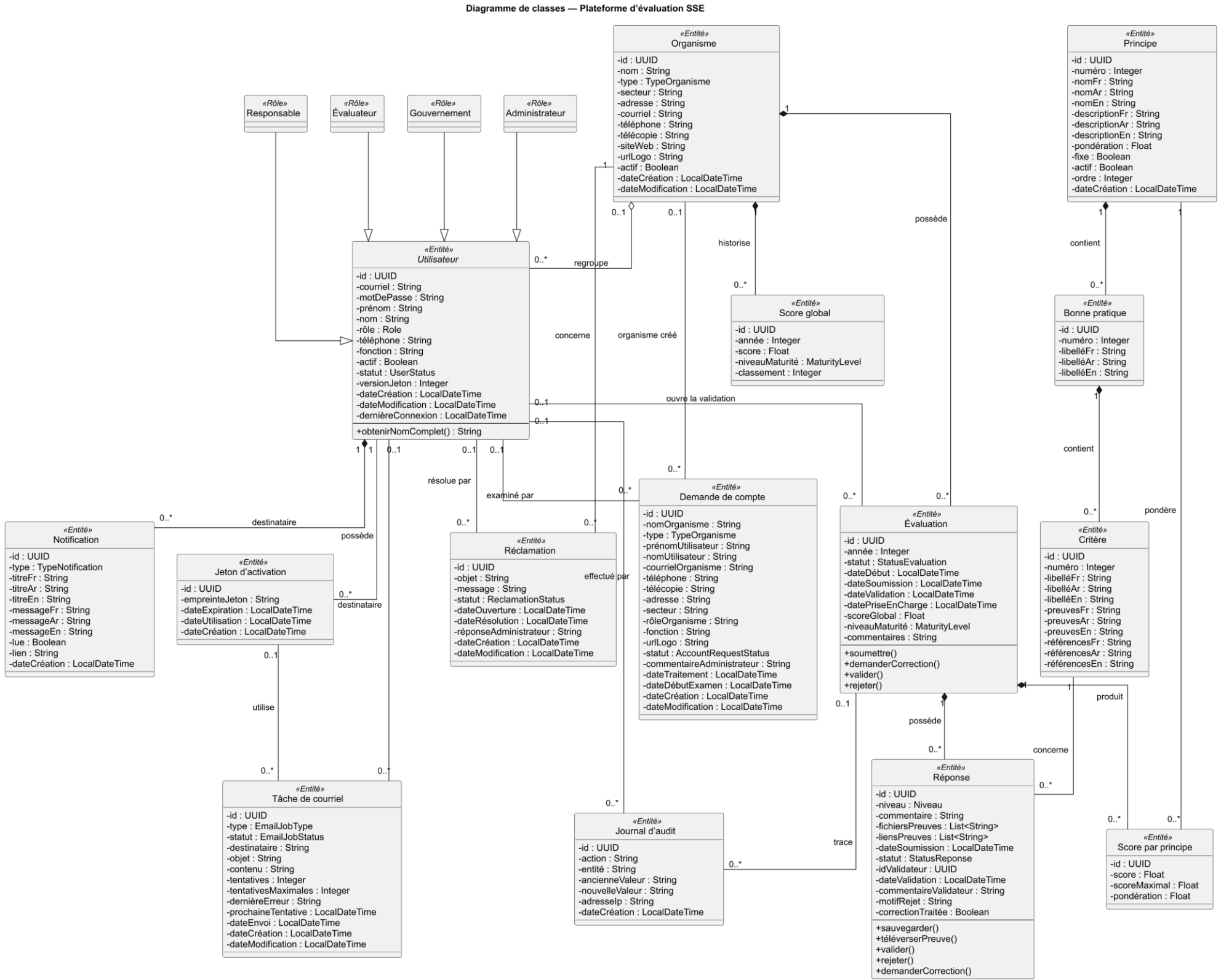


Figure 3 — Diagramme de classes officiel : entités, attributs, généralisations et cardinalités.

3.4 Cycle de vie des évaluations

État	Signification	Transitions principales
EN_COURS	L'organisme renseigne ou corrige son dossier.	SOUMISE après contrôle de complétude.
SOUMISE	Le dossier est disponible pour examen.	EN_VALIDATION lors de la prise en charge ; EN_COURS si correction.
EN_VALIDATION	Un évaluateur possède le verrou de travail.	VALIDEE, REJETEE ou EN_COURS selon la décision.
VALIDEE	Les décisions sont complètes et les scores enregistrés.	État terminal pour l'année concernée.
REJETEE	Le dossier est refusé avec un motif.	État terminal ou reprise selon une future règle métier.

Les réponses possèdent un cycle plus fin : BROUILLON, SOUMISE, VALIDEE, REJETEE ou A_CORRIGER. Le statut A_CORRIGER conserve le commentaire de l'évaluateur et un indicateur signale si l'utilisateur a traité la correction. La validation finale reste bloquée tant qu'une correction n'est pas retournée au responsable ou qu'une décision manque.

3.5 Calcul des scores et de la maturité

Chaque niveau de réponse est converti en points : N0 = 0, N1 = 1, N2 = 2 et N3 = 3. Pour un principe contenant n réponses, le score en pourcentage est calculé par la formule suivante :

FORMULE Score du principe = $(\sum \text{points obtenus} \div (\text{nombre de critères} \times 3)) \times 100$

Le dénominateur est dynamique : il dépend du nombre réel de critères rattachés au principe. Une réponse sans niveau vaut zéro point mais reste comprise dans le dénominateur, ce qui évite de surévaluer une section incomplète. Les principes sans réponse ne modifient pas le score global. Chaque score de principe est ensuite multiplié par sa pondération effective, puis la moyenne pondérée produit le score global.

Intervalle	Niveau de maturité
$0 \leq \text{score} < 25$	INITIAL
$25 \leq \text{score} < 50$	EN_PROGRESSION
$50 \leq \text{score} < 75$	AVANCE
$75 \leq \text{score} \leq 100$	EXCELLENT

3.6 Diagrammes de séquence

Les séquences suivantes décomposent séparément les quatre scénarios les plus importants. Elles font apparaître l'interface web, les contrôleurs, les services métier, le stockage et la base de données.

SÉQUENCE 1 — REMPLIR UNE ÉVALUATION

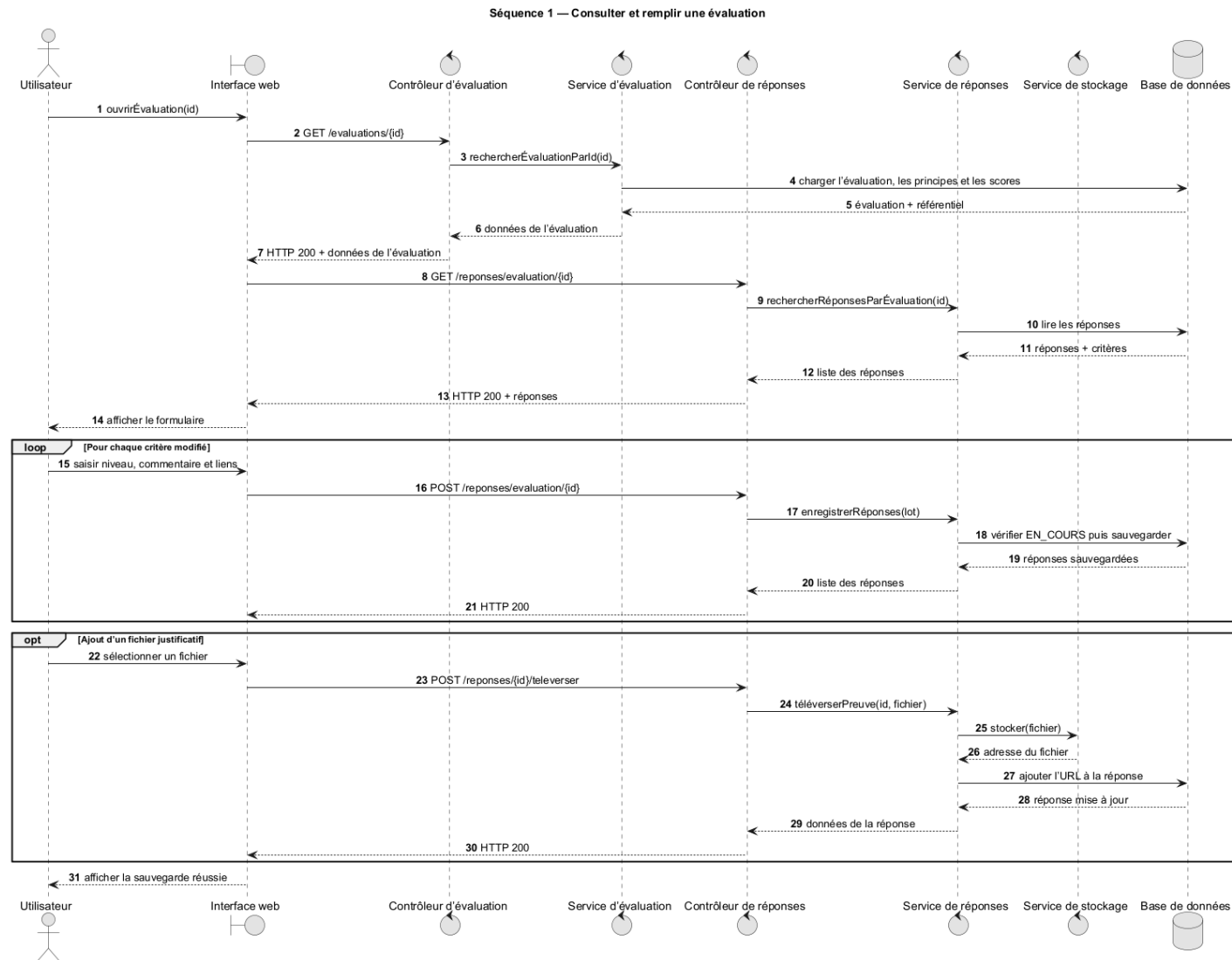
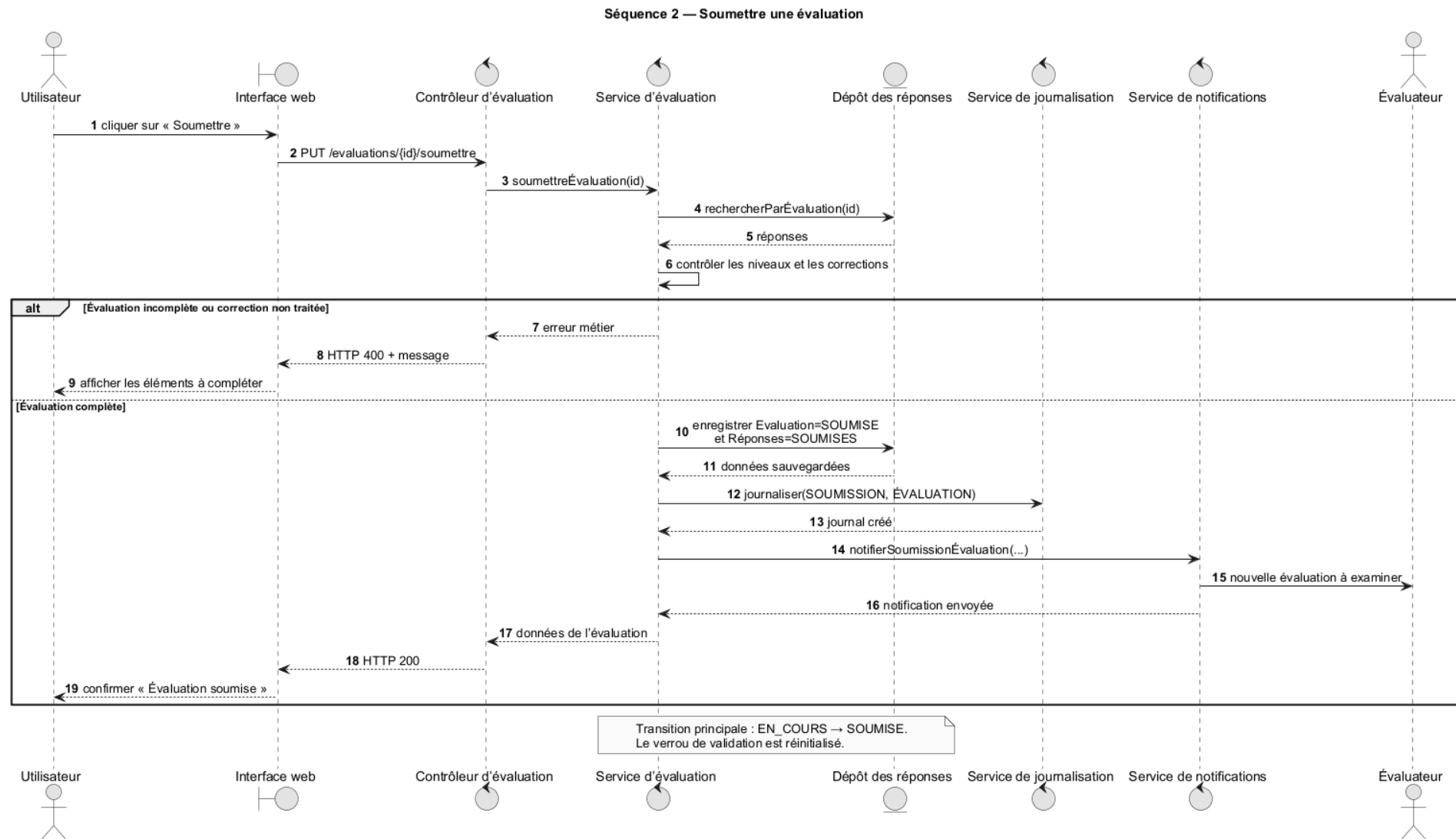


Figure 4 — Chargement, sauvegarde progressive et ajout d'un fichier justificatif.

SÉQUENCE 2 — SOUMETTRE UNE ÉVALUATION



SÉQUENCE 3 — VALIDER UNE ÉVALUATION

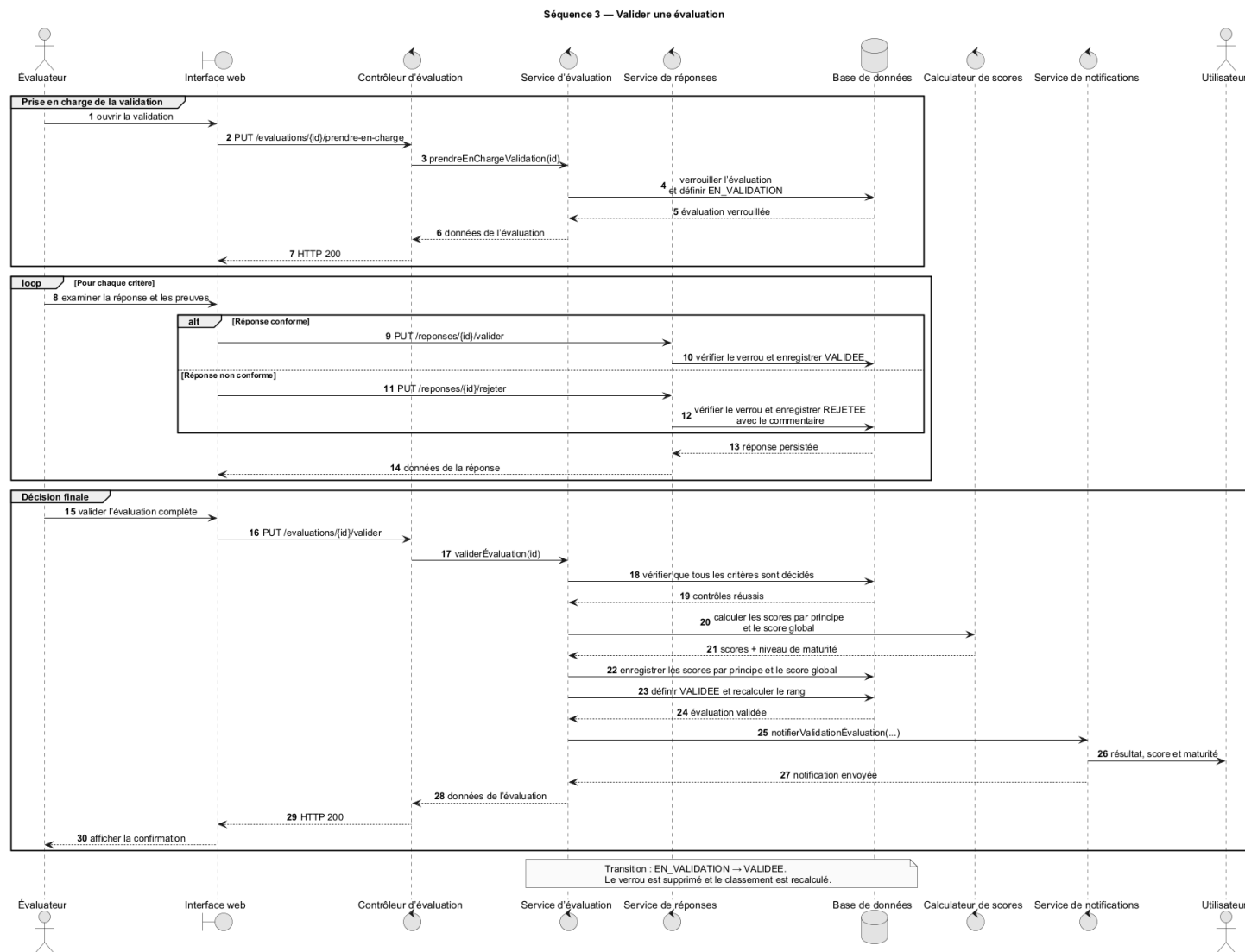


Figure 6 — Prise en charge, décisions par critère, scoring et notification.

SÉQUENCE 4 — CORRIGER ET RESOUMETTRE

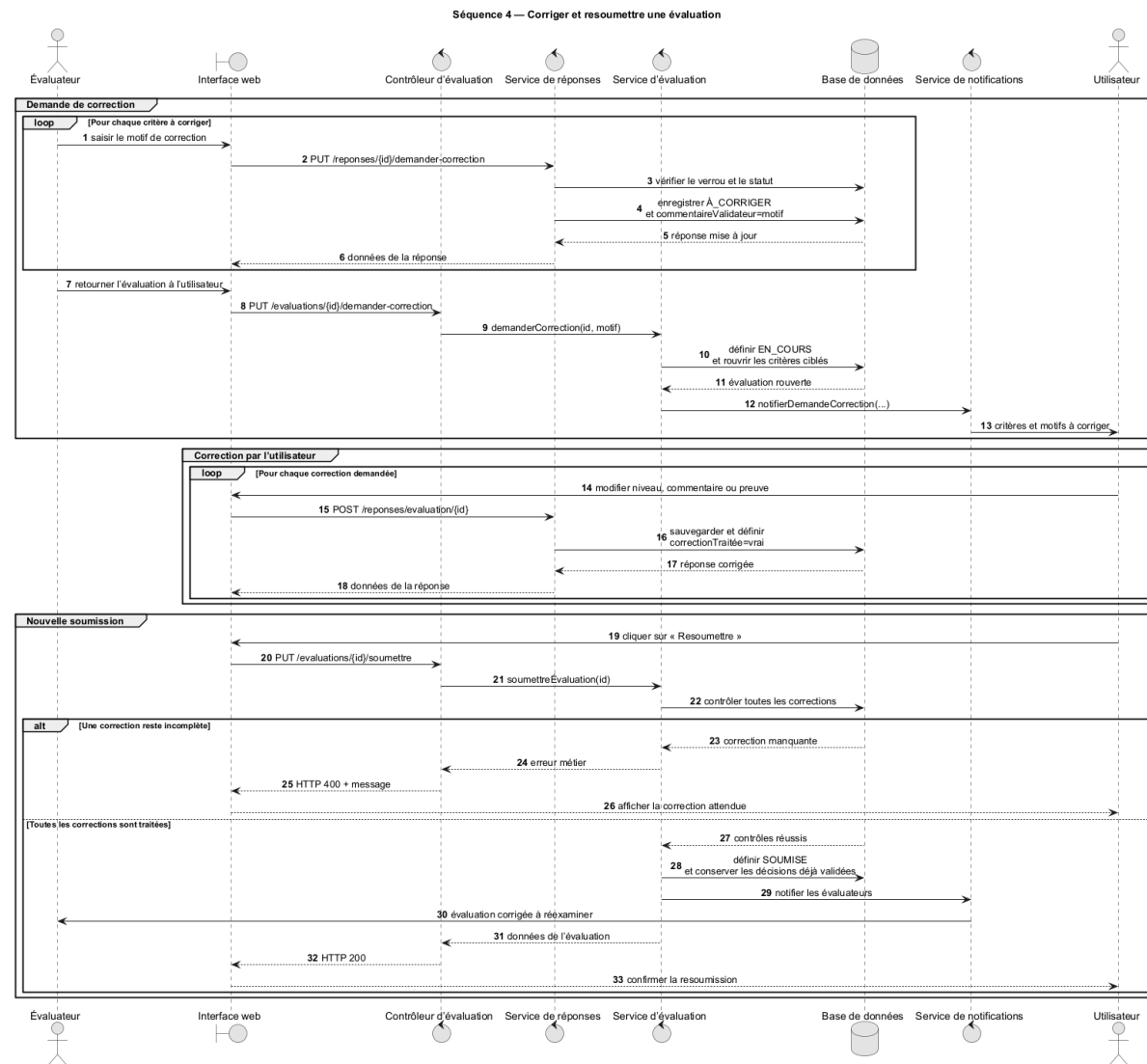


Figure 7 — Correction ciblée, réouverture et nouvelle soumission.

3.7 Choix de conception complémentaires

- un verrou de validation évite deux décisions concurrentes sur le même dossier ;
- les notifications sont persistées et peuvent aussi être diffusées en temps réel ;
- les tâches de courriel mémorisent les tentatives et la dernière erreur afin de faciliter la reprise ;
- les fichiers sont séparés des données relationnelles et référencés par la réponse concernée ;
- les libellés multilingues sont portés par le référentiel, tandis que la préférence de langue appartient au compte ;
- les exports sont générés côté serveur pour garantir une source de données et un rendu homogènes.

Choix	Pourquoi ce choix est important
Verrou applicatif	Évite des décisions contradictoires et rend visible la personne qui traite le dossier.
Correction par critère	Préserve le travail déjà validé et donne à l'organisme une consigne précise.
Scoring côté serveur	Garantit le même résultat quel que soit le navigateur ou l'écran utilisé.
Audit et notifications	Explique les changements d'état et informe le bon acteur au bon moment.
Volumes persistants	Protège les données et les justificatifs lors du remplacement d'un conteneur.

Ces décisions ne sont pas seulement techniques. Elles rendent le processus compréhensible pour les utilisateurs et facilitent le diagnostic lorsqu'un dossier reste bloqué ou qu'une action est contestée.

Chapitre 4 — Réalisation et améliorations apportées

Ce chapitre présente les technologies effectivement utilisées et les contributions réalisées pendant le stage. L'objectif n'a pas été de modifier seulement l'apparence : les évolutions associent règles métier, sécurité, qualité des données et expérience utilisateur.

4.1 Environnement technique

Composant	Technologies	Rôle
Front-end	React 18, TypeScript 5, Vite 5, Tailwind CSS	Interface, routage, responsive design et construction des ressources.
État et formulaires	Zustand, React Hook Form, Zod	État partagé, saisie, validation et messages d'erreur.
Visualisation et i18n	Recharts, i18next, Lucide	Graphiques, traductions FR/AR/EN et iconographie.
Back-end	Java 17, Spring Boot 3.2.5	API REST, services métier, transactions et configuration.
Sécurité	Spring Security, JWT	Authentification JWT et autorisations.
Persistance	Spring Data JPA, PostgreSQL 15	Modèle relationnel, dépôts et requêtes.
Documents	OpenPDF, Apache POI	Génération de rapports PDF et exports Excel.
Déploiement	Docker Compose, Nginx, Ubuntu, AWS EC2	Conteneurisation, exposition web et hébergement.

4.2 Réalisation du back-end

4.2.1 API et services métier

Les contrôleurs sont répartis par domaine : authentification, utilisateurs, organismes, évaluations, réponses, principes, fichiers, notifications, tableaux de bord, rapports, réclamations, demandes de compte, audit et tâches de courriel. Cette organisation permet de limiter les responsabilités de chaque point d'entrée.

La logique complexe réside dans les services. Le service d'évaluation vérifie les statuts, prend ou libère le verrou de validation, calcule les scores, met à jour le classement et déclenche les notifications. Le service de réponses contrôle les modifications autorisées, les décisions de l'évaluateur et le traitement des corrections. Les opérations sont transactionnelles afin que la mise à jour des statuts, scores et historiques reste atomique.

4.2.2 Sécurisation des ressources

Un composant de contrôle d'accès complète les annotations de rôle. Il vérifie, par exemple, que l'utilisateur connecté appartient bien à l'organisme de l'évaluation avant d'autoriser la lecture ou l'écriture. Le même principe est appliqué au téléversement d'une preuve. Les évaluateurs et administrateurs disposent des actions de validation, tandis que le gouvernement accède aux vues de pilotage.

4.2.3 Notifications, audit et documents

Les événements majeurs produisent des notifications : nouvelle soumission, correction demandée, validation ou rejet. Les notifications sont sauvegardées, affichées dans l'interface et diffusables via Server-Sent Events. Le journal d'audit conserve l'action, l'entité, les valeurs avant/après, l'utilisateur et l'adresse IP

lorsque l'information est disponible. Les rapports PDF et fichiers Excel sont générés au back-end pour faciliter l'archivage et le partage.

4.3 Refonte de l'interface et de l'expérience utilisateur

La refonte a établi un langage visuel commun : navigation latérale structurée par rôle, en-têtes cohérents, cartes d'indicateurs, contrastes renforcés, thème clair/sombre, espacements réguliers, icônes compréhensibles et composants réutilisables. Les pages mettent en avant la prochaine action utile plutôt qu'une simple accumulation de données.



Figure 8 — Tableau de bord administrateur : indicateurs, synthèse et navigation institutionnelle.

Le tableau de bord administrateur regroupe les indicateurs essentiels et donne accès aux domaines d'administration. Le mode sombre illustré ci-dessus conserve un contraste lisible et distingue clairement les cartes, graphiques et actions.

4.3.1 Navigation et fermeture des interfaces

- ajout d'une flèche de retour dans les interfaces détaillées et les formulaires afin de revenir à la page précédente ;
- ajout d'une icône « X » pour quitter explicitement les formulaires ou modales ;
- correction du voile de fond des fenêtres : la petite bande blanche supérieure a été supprimée et l'arrière-plan est désormais uniformément assombri ;
- agrandissement du logo d'un organisme au clic depuis le tableau des utilisateurs ou la fiche détaillée ;
- uniformisation des titres de page, du fil d'action et des états vides.

4.3.2 Interface de remplissage et de validation

L'interface d'évaluation devait afficher simultanément les principes, les bonnes pratiques, les critères, la réponse, les preuves et les actions de validation. La mise en page a été rendue adaptable : la zone principale grandit ou se réduit selon la longueur du critère, au lieu de réserver un espace vide fixe au-dessus des boutons. L'espacement entre la flèche de retour et les listes de navigation a également été augmenté.

Dans l'espace évaluateur, la navigation par principe et par critère affiche une progression et des filtres par décision. Les actions Valider, Demander correction et Rejeter restent rattachées au critère visible. La validation globale est désactivée tant que les décisions ne sont pas complètes, ce qui relie le feedback visuel à la règle métier.

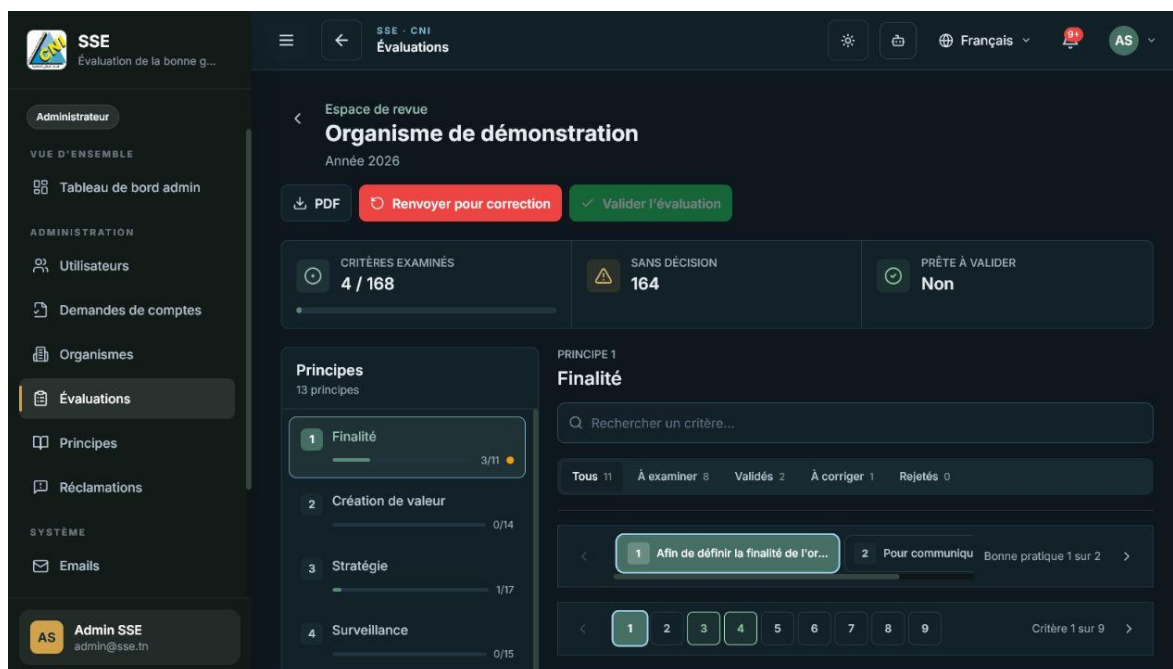


Figure 11 — Interface de revue : progression, principes, critères, preuves et décision de l'évaluateur.

4.3.3 Formulaires administratifs complets

Le formulaire de création d'un utilisateur a été aligné sur la demande publique de compte. Les champs manquants — notamment télécopie, logo, adresse, secteur, rôle de l'organisme et fonction — ont été ajoutés. Les validations et libellés suivent la même terminologie, ce qui réduit les corrections manuelles après création.

4.3.4 Interface adaptée aux rôles

Le menu et le tableau de bord ne présentent que les fonctions utiles. La section d'évaluation a été retirée du profil gouvernement, puisque ce rôle ne doit ni remplir ni examiner les dossiers. Il conserve uniquement les indicateurs et le classement. Le profil utilisateur se concentre sur ses évaluations et le référentiel. L'évaluateur dispose d'une file de validation. L'administrateur conserve la vue globale.

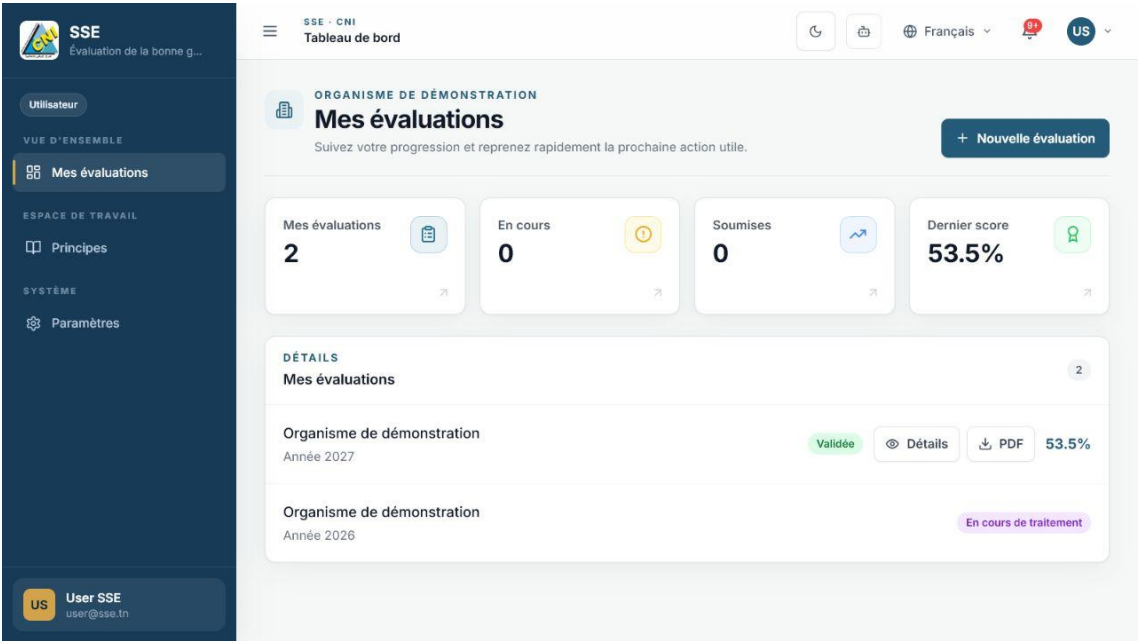


Figure 9 — Tableau de bord utilisateur : progression et actions liées à son organisme.

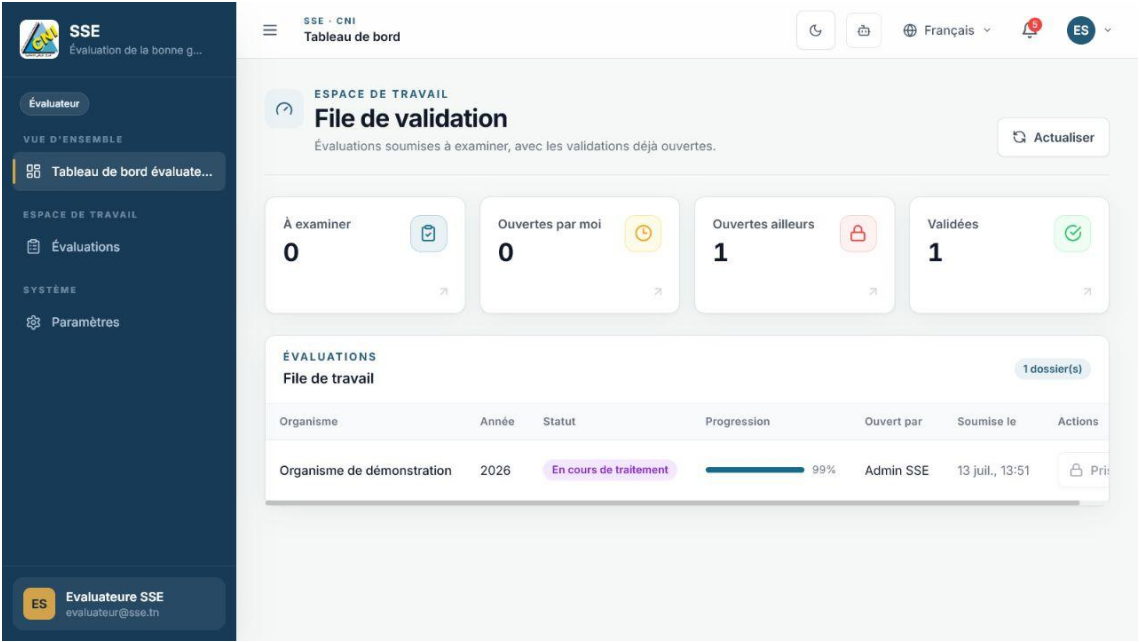


Figure 10 — Tableau de bord évaluateur : file de travail, progression et prise en charge.

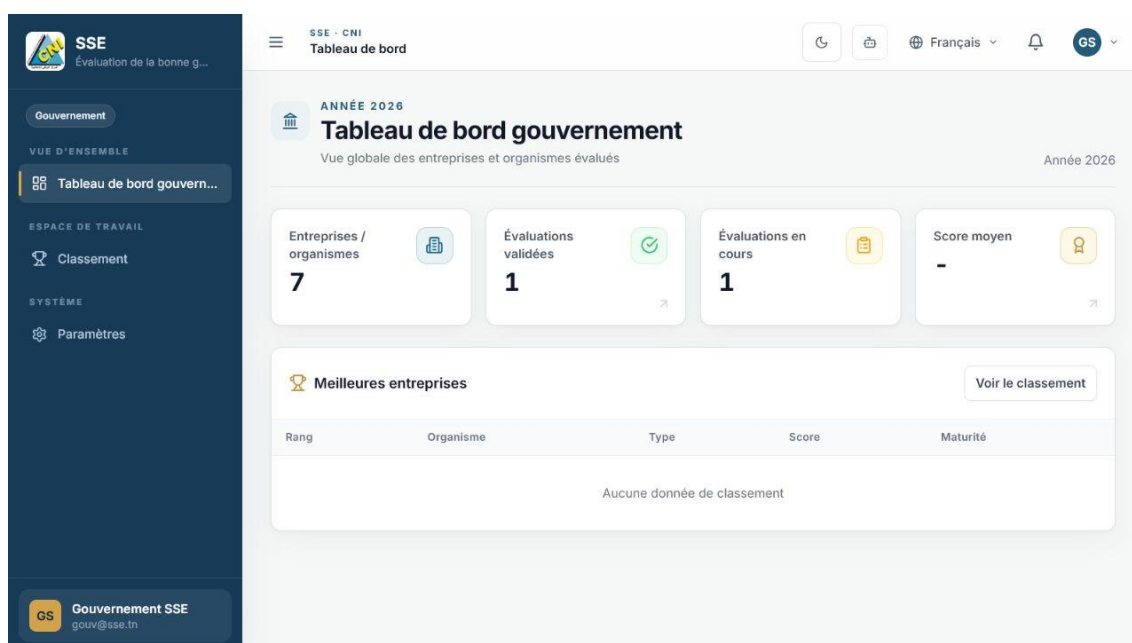


Figure 12 — Tableau de bord gouvernemental : indicateurs consolidés sans section de saisie.

4.4 Internationalisation propre à chaque compte

Une préférence globale enregistrée uniquement dans le navigateur provoquait un défaut : après avoir choisi l'arabe avec un compte évaluateur, le compte administrateur ouvert ensuite héritait de la même langue. La préférence est désormais associée à l'identité du compte. Lors de la connexion, l'application charge la langue enregistrée pour cet utilisateur ; lors d'un changement, elle met à jour uniquement cette préférence.

Le français, l'anglais et l'arabe sont disponibles. L'arabe active la direction droite-à-gauche, ce qui concerne l'alignement, la navigation et l'ordre visuel de plusieurs composants. Le changement de langue reste accessible depuis l'en-tête, sans mélanger les choix de deux sessions successives.

4.5 Paramètres utiles

L'interface Paramètres a été conservée mais recentrée sur des options réellement utiles : thème visuel, langue, préférences de notification et informations du profil lorsque l'autorisation le permet. Cette approche évite une page vide tout en regroupant les réglages personnels hors des écrans métier.

4.6 Qualité d'usage

Problème initial	Amélioration	Effet utilisateur
Grand espace vide dans la validation	Hauteur de contenu adaptative.	Les actions restent proches du critère quelle que soit sa longueur.
Langue partagée entre comptes	Clé de préférence liée au compte.	Chaque utilisateur retrouve son dernier choix.
Bande blanche au-dessus des modales	Overlay fixé sur toute la fenêtre.	Concentration et cohérence visuelle.
Retour incertain	Flèche de retour et bouton de fermeture.	Parcours plus réversible et moins d'erreurs.
Logos trop petits	Aperçu agrandi au clic.	Vérification visuelle facilitée.
Fonctions non pertinentes pour le gouvernement	Navigation filtrée par rôle.	Interface plus simple et conforme aux droits.
Formulaire utilisateur incomplet	Ajout des champs de l'organisme.	Création sans étape corrective supplémentaire.

Ces corrections ont été vérifiées sur les écrans réellement déployés. J'ai particulièrement contrôlé les passages d'un rôle à l'autre, car plusieurs défauts n'apparaissaient qu'après une déconnexion ou après l'ouverture successive de comptes différents dans le même navigateur.

RÉSULTAT L'interface conserve une identité commune, mais chaque profil voit un parcours plus court, centré sur ses responsabilités et sur la prochaine action utile.

Chapitre 5 — Tests, déploiement et exploitation

Une fonctionnalité n'est réellement terminée que si son comportement peut être vérifié dans un environnement proche de l'usage réel. Les contrôles ont donc porté sur les règles métier, les API, la construction du front-end, le rendu des interfaces et le déploiement conteneurisé.

5.1 Stratégie de test

La stratégie combine plusieurs niveaux. Les tests unitaires isolent une règle, comme le calcul dynamique des scores. Les tests de services utilisent des dépendances simulées pour vérifier l'authentification ou les indicateurs. Les tests de contrôleur valident le contrat HTTP des rapports. Enfin, les parcours fonctionnels sont contrôlés dans le navigateur avec les comptes de démonstration des quatre rôles.

Type	Éléments contrôlés	Exemples
Unitaire	Calcul pur et cas limites.	Dénominateur selon le nombre de critères, réponse vide, score maximal.
Service	Règles métier avec dépôts simulés.	Connexion, tableau de bord, transitions et autorisations.
Contrôleur	Statut HTTP, contenu et en-têtes.	Téléchargement d'un rapport PDF.
Construction	Compilation et dépendances.	Maven pour le back-end, TypeScript/Vite pour le front-end.
Fonctionnel	Parcours de bout en bout.	Connexion par rôle, navigation, revue, langue, modales et classement.
Visuel	Responsive design et cohérence.	Espace adaptatif, retour, voile de modale, thème clair/sombre.

5.2 Cas de test critiques

ID	Scénario	Résultat attendu
T-01	Connexion avec chacun des quatre rôles.	Redirection vers le tableau de bord autorisé et menu adapté.
T-02	Un utilisateur tente d'accéder à l'évaluation d'un autre organisme.	Accès refusé par le contrôle de propriété.
T-03	Soumission avec un critère requis incomplet.	Soumission refusée et message ciblé.
T-04	Deux évaluateurs ouvrent le même dossier.	Un seul verrou actif ; le second voit le dossier pris.
T-05	Validation globale avec une décision manquante.	Bouton désactivé ou erreur métier ; aucun score final enregistré.
T-06	Critère renvoyé pour correction.	Seul le critère concerné est rouvert avec son motif.
T-07	Changement de langue puis connexion avec un autre compte.	Chaque compte retrouve sa propre préférence.
T-08	Ouverture d'une modale sur une page longue.	Voile couvrant toute la fenêtre, sans bande blanche.
T-09	Clic sur un logo depuis une liste ou une fiche.	Aperçu agrandi et refermable.
T-10	Accès gouvernemental.	Indicateurs et classement disponibles ; aucune interface de saisie.

5.3 Conteneurisation

Docker Compose regroupe trois services principaux. PostgreSQL 15 utilise un volume persistant pour les données. Le back-end Spring Boot communique avec la base par le réseau Docker et expose l'API. Le front-end est compilé puis servi par Nginx, qui distribue les ressources et transmet les requêtes API. Un second volume conserve les justificatifs envoyés.

- des healthchecks vérifient la disponibilité de la base et du back-end ;
- les dépendances entre services évitent de démarrer l'application avant PostgreSQL ;
- les paramètres sensibles sont injectés par variables d'environnement ;
- les volumes séparent les données durables du cycle de vie des conteneurs ;
- la même composition peut être utilisée localement et adaptée en production.

5.4 Déploiement sur Ubuntu dans AWS EC2

La version finale a été construite et déployée sur une machine Ubuntu hébergée dans Amazon EC2. EC2 fournit une machine virtuelle configurable dans le cloud AWS [7]. L'accès d'administration se fait par SSH à l'aide d'une clé privée conservée hors du dépôt. Pour des raisons de sécurité, l'adresse publique et le chemin local de cette clé ne sont pas reproduits dans ce rapport.

Élément	Configuration retenue
Système hôte	Ubuntu sur une instance Amazon EC2.
Orchestration	Docker Compose pour le front-end, le back-end et PostgreSQL.
Exposition	Nginx sert l'application React et transmet les requêtes vers l'API.
Persistance	Volume PostgreSQL et volume séparé pour les justificatifs.
Configuration	Variables d'environnement pour la base, JWT, CORS, SMTP et options de production.
Vérification	Healthchecks, journaux des conteneurs et tests des quatre rôles.

Pendant la mise en ligne, j'ai volontairement séparé les informations nécessaires au rapport des éléments secrets d'exploitation. Le rapport décrit donc l'architecture et la procédure, mais n'expose ni clé privée ni adresse sensible de connexion.

ARCHITECTURE DE DÉPLOIEMENT

Architecture de déploiement de la plateforme SSE

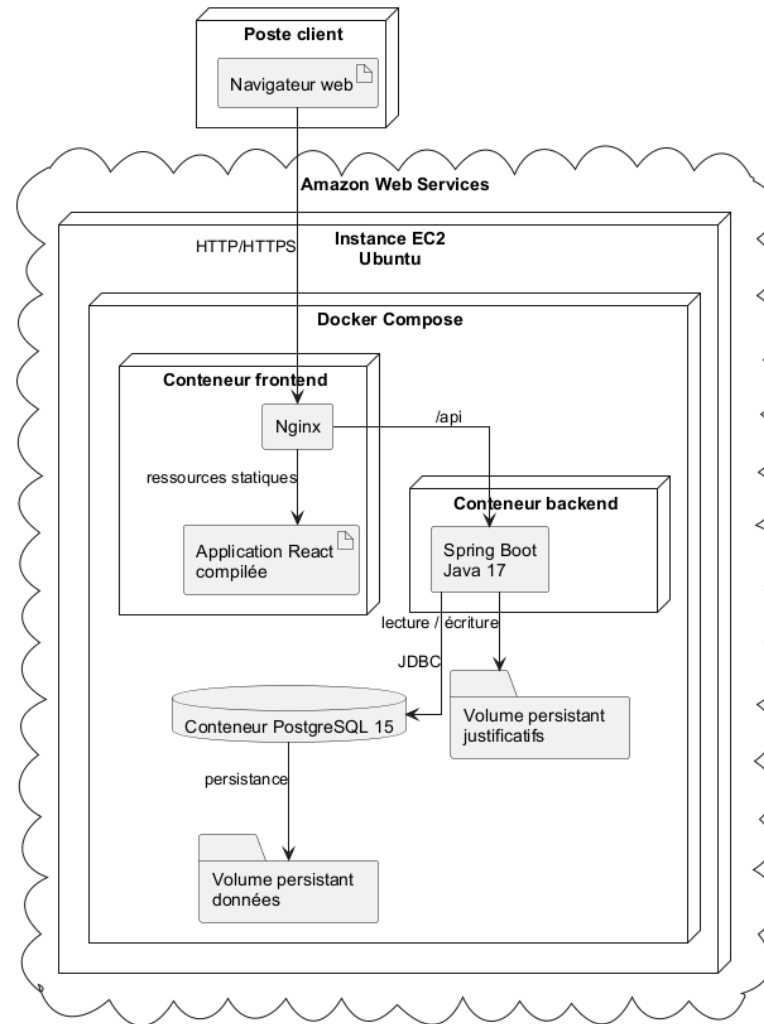


Figure 13 — Déploiement Docker Compose sur une instance Ubuntu dans AWS EC2.

5.4.1 Procédure de mise en production

1. sauvegarder l'état validé du dépôt dans Git et identifier le commit de référence ;
2. transférer ou récupérer le code sur l'instance Ubuntu ;
3. renseigner les variables d'environnement de production sans committer les secrets ;
4. construire les images et démarrer les services avec Docker Compose ;
5. vérifier les healthchecks, les journaux et la connexion à PostgreSQL ;
6. tester la connexion, les rôles, la navigation et un téléchargement ;
7. conserver un plan de retour vers l'image ou le commit précédent.

5.4.2 Sauvegarde et retour arrière

Avant la refonte UI/UX majeure, l'état courant a été conservé dans l'historique Git afin de pouvoir revenir à une version connue. Le rollback applicatif consiste à redéployer le commit ou l'image précédente. Le rollback de données doit rester distinct : il nécessite une sauvegarde PostgreSQL et une copie des volumes de justificatifs avant toute migration susceptible de modifier le schéma.

RÈGLE D'EXPLOITATION Une sauvegarde du code ne remplace jamais une sauvegarde des données. Le dépôt Git protège les sources ; les volumes et les exports PostgreSQL protègent l'information métier.

5.5 Sécurité d'exploitation

- restreindre le groupe de sécurité EC2 aux ports réellement nécessaires ;
- protéger la clé SSH, ne jamais l'envoyer dans Git et limiter ses permissions ;
- placer l'application derrière HTTPS avec un nom de domaine et un certificat TLS ;
- renouveler les secrets JWT et les mots de passe de démonstration avant un usage réel ;
- désactiver ou supprimer les jeux de données de test dans l'environnement de production ;
- planifier les sauvegardes de PostgreSQL et du volume des justificatifs ;
- surveiller l'espace disque, les healthchecks, les erreurs d'API et la file de courriels.

5.6 Résultats de vérification

Les interfaces déployées ont été parcourues avec les quatre comptes de démonstration. Les tableaux de bord correspondent aux responsabilités attendues : administration complète, espace d'évaluation de l'organisme, file de travail de l'évaluateur et indicateurs gouvernementaux. Les captures intégrées au chapitre précédent proviennent de cette vérification et non de maquettes statiques.

Les tests automatisés du dépôt couvrent le calcul des scores, l'authentification, le service de tableau de bord et le contrôleur de rapports. Lors de la finalisation du rapport, la construction de production du front-end a été exécutée avec succès. La construction complète et la suite Maven doivent rester des contrôles obligatoires avant chaque livraison.

Chapitre 6 — Bilan et perspectives

Le stage a permis de travailler sur une application institutionnelle complète, depuis la compréhension du métier jusqu'au déploiement. Il a également montré que la qualité d'un produit ne dépend pas d'un seul écran ou d'une seule technologie, mais de la cohérence entre données, règles, sécurité, interface et exploitation.

6.1 Difficultés rencontrées et solutions

Difficulté	Analyse	Solution retenue
Workflow multi-états	Une action sur une réponse influence l'état global de l'évaluation.	Centralisation des transitions dans les services et contrôles avant validation.
Validation concurrente	Deux évaluateurs pouvaient agir sur le même dossier.	Prise en charge avec propriétaire du verrou et libération contrôlée.
Corrections ciblées	Rouvrir tout le dossier faisait perdre le contexte.	Statut A_CORRIGER, motif par critère et indicateur correction traitée.
Interface dense	Les listes et actions occupent beaucoup d'espace.	Mise en page responsive, hauteur adaptative, progression et filtres.
Préférence de langue	Le stockage navigateur était partagé entre comptes.	Clé de préférence liée à l'identité et restauration à la connexion.
Déploiement	Différences entre poste local et serveur Ubuntu.	Docker Compose, variables d'environnement et contrôles de santé.

6.2 Compétences acquises

L'un des principaux enseignements du stage a été d'accepter qu'un problème visible n'a pas toujours une cause visible. Une simple préférence de langue pouvait venir du stockage de session ; un bouton désactivé pouvait dépendre d'une règle de complétude ; un écran dense pouvait révéler une mauvaise hiérarchie de l'information. Cette recherche de la cause avant la correction est la compétence que je retiens le plus.

- analyser un dépôt existant sans perdre les changements déjà réalisés ;
- traduire une règle métier en états, autorisations et transactions ;
- modéliser une solution avec des diagrammes de cas d'utilisation, de classes et de séquence ;
- développer et structurer une API REST avec Spring Boot ;
- construire une interface React responsive, accessible et multilingue ;
- écrire des tests ciblés et diagnostiquer un comportement dans l'environnement déployé ;
- utiliser Git comme mécanisme de traçabilité et de sauvegarde du code ;
- conteneuriser et déployer une application full-stack sur Ubuntu dans AWS EC2 ;
- documenter des choix techniques et présenter un résultat à des publics différents.

6.3 Apports personnels au projet

Mes contributions couvrent l'ensemble du produit. Sur le plan fonctionnel, j'ai clarifié les rôles et consolidé les parcours de soumission, prise en charge, correction et validation. Sur le plan technique, j'ai renforcé les contrôles d'accès, la logique de scoring, les notifications, les exports et l'exploitation. Sur le plan visuel, j'ai mené une refonte globale et corrigé des problèmes précis signalés pendant les essais. Chaque fois que cela

était possible, j'ai vérifié le résultat directement dans l'application déployée plutôt que de m'arrêter au code compilé.

- navigation arrière généralisée et fermeture explicite des formulaires ;
- interface de revue adaptative et actions rapprochées du critère ;
- préférence de langue isolée par compte ;
- correction complète du voile des modales ;
- agrandissement des logos au clic ;
- suppression de la saisie d'évaluation pour le rôle gouvernement ;
- complétion du formulaire de création d'utilisateur avec les données de l'organisme ;
- tableaux de bord et navigation adaptés aux quatre rôles ;
- construction, sauvegarde Git et déploiement de la version améliorée.

6.4 Limites actuelles

La version réalisée constitue une base solide, mais plusieurs éléments doivent être renforcés avant un usage institutionnel à grande échelle. Le déploiement de démonstration contient encore des comptes et données de test. La couverture automatisée reste partielle. La supervision, les sauvegardes planifiées et la gestion fine du cycle de vie des fichiers doivent être industrialisées.

6.5 Perspectives d'évolution

Priorité	Évolution proposée	Bénéfice
Court terme	Activer HTTPS, remplacer les secrets et retirer toutes les données de démonstration.	Sécuriser la mise en service réelle.
Court terme	Ajouter des tests end-to-end sur les parcours des quatre rôles.	Détecter les régressions de workflow et d'interface.
Court terme	Automatiser sauvegardes PostgreSQL et volumes.	Réduire le risque de perte de données.
Moyen terme	Mettre en place une intégration et un déploiement continu.	Rendre les livraisons reproductibles et traçables.
Moyen terme	Stocker les justificatifs dans un service objet compatible S3.	Améliorer la durabilité, la capacité et la distribution des fichiers.
Moyen terme	Ajouter une supervision centralisée et des alertes.	Réagir rapidement aux erreurs et dégradations.
Long terme	Enrichir les analyses comparatives et tendances pluriannuelles.	Renforcer le pilotage gouvernemental.
Long terme	Réaliser un audit d'accessibilité et un test de charge.	Préparer un usage large et inclusif.

Conclusion générale

Le stage effectué au Centre National de l'Informatique a abouti à la consolidation et au déploiement d'une plateforme complète de suivi et d'évaluation de la bonne gouvernance. Le projet répond à un besoin exigeant : guider la saisie d'un référentiel détaillé, associer des preuves à chaque réponse, organiser une revue traçable, traiter les corrections et produire des résultats consolidés.

La solution s'appuie sur une architecture moderne et maintenable : React et TypeScript pour l'interface, Spring Boot pour l'API et la logique métier, PostgreSQL pour les données, Docker Compose pour l'exécution et AWS EC2 pour l'hébergement. La sécurité combine authentification JWT, rôles et contrôle de propriété des ressources. Le scoring dynamique et les transitions contrôlées assurent la cohérence du résultat.

Les améliorations réalisées ont également transformé l'expérience utilisateur. Les quatre profils disposent désormais d'un espace mieux adapté à leur mission, d'une navigation réversible, de formulaires complets, d'une préférence de langue propre au compte et d'interfaces plus lisibles. Les défauts visuels signalés ont été corrigés sans dissocier le design des règles métier sous-jacentes.

Au-delà du produit livré, cette expérience m'a appris à travailler de manière méthodique sur une base de code réelle, à justifier les choix, à protéger les données existantes, à vérifier une version déployée et à documenter le résultat. Les perspectives proposées — HTTPS, tests end-to-end, sauvegardes automatisées, CI/CD et supervision — constituent les prochaines étapes vers une exploitation institutionnelle durable.

Bibliographie et webographie

Les ressources suivantes ont été consultées pour documenter le contexte institutionnel et les technologies.
Date de consultation des ressources en ligne : 22 juillet 2026.

- [1] Centre National de l'Informatique, « Présentation du CNI ». <https://www.cni.tn/index.php/fr/layout-3/presentation-du-cni-2>
- [2] Centre National de l'Informatique, « Formation ». <https://www.cni.tn/index.php/fr/pages-3/formation>
- [3] ESPRIT, site institutionnel et présentation de l'approche pédagogique. <https://www.esprit.tn/>
- [4] VMware, Spring Boot Reference Documentation, version 3.2. <https://docs.spring.io/spring-boot/>
- [5] Meta Open Source, React Documentation, « Describing the UI ». <https://react.dev/learn/describing-the-ui>
- [6] Docker, « Docker Compose documentation ». <https://docs.docker.com/compose/>
- [7] Amazon Web Services, « Amazon EC2 Documentation ». <https://docs.aws.amazon.com/ec2/>
- [8] PostgreSQL Global Development Group, PostgreSQL Documentation. <https://www.postgresql.org/docs/>
- [9] OpenAPI Initiative, « OpenAPI Specification ». <https://spec.openapis.org/oas/latest.html>
- [10] OWASP Foundation, « JSON Web Token Cheat Sheet for Java ». https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html

Annexes

CONTENU Les annexes regroupent les principaux points d'entrée de l'API, les transitions de statut, une procédure d'installation et un glossaire métier.

Annexe A — Principaux points d'entrée REST

Domaine	Méthode et route	Usage
Authentification	POST /auth/login	Connexion et émission des jetons.
Authentification	POST /auth/refresh	Renouvellement du jeton d'accès.
Comptes	POST /account-requests	Dépôt public d'une demande de compte.
Administration	GET /admin/users	Liste paginée des utilisateurs.
Administration	POST /admin/users/with-organisme	Création d'un utilisateur et de son organisme.
Organismes	GET /organismes/{id}	Consultation autorisée d'un organisme.
Référentiel	GET /principes	Lecture des principes, bonnes pratiques et critères.
Évaluations	POST /evaluations	Création d'une évaluation.
Évaluations	PUT /evaluations/{id}/submit	Soumission après contrôle de complétude.
Évaluations	PUT /evaluations/{id}/claim	Prise en charge de la validation.
Évaluations	PUT /evaluations/{id}/validate	Validation finale et calcul du score.
Évaluations	PUT /evaluations/{id}/request-correction	Retour ciblé vers l'utilisateur.
Réponses	POST /reponses/evaluation/{id}	Sauvegarde d'un lot de réponses.
Réponses	PUT /reponses/{id}/validate	Validation d'une réponse.
Réponses	PUT /reponses/{id}/reject	Rejet d'une réponse avec motif.
Fichiers	POST /files ou route de téléversement	Ajout d'un justificatif autorisé.
Notifications	GET /notifications	Liste et état de lecture.
Rapports	GET /reports/evaluations/{id}/pdf	Téléchargement du rapport PDF.
Pilotage	GET /dashboard/...	KPI, distributions et classement.

Annexe B — Matrice des transitions

État initial	Action	Acteur	État final
EN_COURS	Soumettre	Utilisateur / administrateur autorisé	SOUMISE
SOUMISE	Prendre en charge	Évaluateur / administrateur	EN_VALIDATION
SOUMISE ou EN_VALIDATION	Demander correction	Propriétaire du verrou	EN_COURS
SOUMISE ou EN_VALIDATION	Valider toutes les réponses et le dossier	Propriétaire du verrou	VALIDEE

État initial	Action	Acteur	État final
SOUMISE EN_VALIDATION	ou Rejeter le dossier	Propriétaire du verrou	REJETEE
EN_VALIDATION	Libérer ou expirer le verrou	Système / propriétaire	SOUMISE

Annexe C — Installation locale synthétique

1. installer Git et Docker avec le module Docker Compose ;
2. cloner le dépôt puis se placer à sa racine ;
3. copier le fichier d'exemple des variables d'environnement et définir les secrets locaux ;
4. démarrer l'ensemble avec la commande Docker Compose du projet ;
5. attendre que PostgreSQL et le back-end soient déclarés sains ;
6. ouvrir l'adresse du front-end, puis vérifier la connexion avec un compte de test ;
7. consulter les journaux des services si un healthcheck échoue.

```
docker compose up -d --build
docker compose ps
docker compose logs --tail=200 backend
```

Annexe D — Glossaire métier

Terme	Définition
Principe	Axe majeur du référentiel de gouvernance.
Bonne pratique	Ensemble cohérent de pratiques rattachées à un principe.
Critère	Question ou exigence élémentaire qui reçoit un niveau, un commentaire et des preuves.
Réponse	État renseigné par l'organisme pour un critère donné dans une évaluation.
Justificatif	Fichier ou lien permettant d'étayer une réponse.
Évaluation	Dossier annuel d'un organisme couvrant le référentiel actif.
Validateur	Évaluateur ou administrateur autorisé qui examine le dossier.
Maturité	Catégorie synthétique dérivée du score global.
Verrou	Mécanisme qui réserve temporairement une validation à un utilisateur.
Audit	Trace horodatée d'une opération sensible.

Annexe E — Livrables UML

Les fichiers sources et les exports des diagrammes de classes, de cas d'utilisation et de séquence sont conservés avec le projet. Les deux diagrammes officiels fournis pour ce rapport ont été convertis depuis SVG vers PNG haute résolution sans modification de leur contenu. Les diagrammes de séquence sont décomposés en quatre scénarios indépendants afin de faciliter leur lecture et leur présentation.

Annexe F — Checklist avant mise en production

- supprimer les comptes, principes et données de démonstration ;
- activer HTTPS et contrôler les redirections ;
- remplacer tous les secrets et vérifier qu'aucun n'est suivi par Git ;
- tester les quatre rôles avec des comptes de recette ;
- exécuter la suite Maven et la construction Vite ;
- sauvegarder PostgreSQL et les justificatifs ;
- vérifier les limites de taille et types de fichiers ;
- contrôler le journal d'audit, la file de courriels et les notifications ;
- préparer la procédure de rollback et identifier le commit déployé ;
- documenter le responsable d'exploitation et le canal de support.